Apache Spark and Resilient Distributed Datasets (RDDs)

Big Data Processing Distributed Computing Functional Programming

November 5, 2025

Apache Spark Documentation

Introduction to Apache Spark

Fast Processing Engine

Rapid data processing capability designed for large-scale clusters, enabling high-throughput data analysis.

Distributed Computing

Processes data in parallel across multiple machines, allowing for efficient handling of vast datasets.

General-Purpose Engine

Versatile platform supporting batch processing, streaming, machine learning, and graph processing in a single engine.

Resilient Distributed Datasets

Built-in support for Resilient Distributed Datasets (RDDs), the fundamental data structure for distributed computing.

Apache Spark processes data in parallel across a cluster

Parallel Processing

Fault Tolerance

Recovery

What is an RDD?

Definition

Resilient Distributed Dataset (RDD) is a distributed collection of data that can be processed in parallel across multiple nodes within a cluster.

Core Properties

Resilient

Fault-tolerant with automatic recovery from node failures through lineage tracking.

Distributed

Data is partitioned and distributed across various nodes in a cluster for parallel computation.

Immutable

Once created, RDDs cannot be changed. Transformations create new RDDs, leaving originals unaltered.

Lazy Evaluation

Operations are not executed immediately but built into a logical plan, computed when an action is called.

Practical Example

Transaction Data Analysis

Consider analyzing e-commerce transaction data to identify top-selling products:

```
# Load transaction data into RDD
val rdd = sc.textFile("transactions.txt")
# Transformation: Process each transaction
val processedRDD = rdd.map(line => line.split(","))
# Action: Trigger computation and collect results
val result = processedRDD.collect()
```

Key Insight

The data is not loaded into memory until the action (collect()) is called. This lazy evaluation optimizes the overall computation.

Transformations vs. Actions

Transformations

- Creates a new RDD from an existing one
- "Lazy" evaluation - does not execute immediately
- Builds a logical plan of operations

Examples:

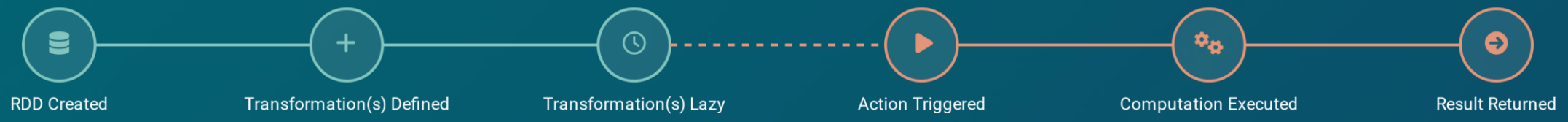
map() filter() flatMap() reduceByKey()

Actions

- Triggers actual computation of RDD transformations
- Returns a result to the driver program
- Writes data to external storage systems

Examples:

collect() count() saveAsTextFile() reduce()



Fault Tolerance and Lineage-DAG

Inherent Fault Tolerance

RDDs provide automatic fault tolerance through a Lineage-DAG (Directed Acyclic Graph) that records all transformations applied to the base data.

Lineage Information

The Lineage-DAG maintains a record of how an RDD was derived from its parent RDDs, enabling precise data recovery when failures occur.

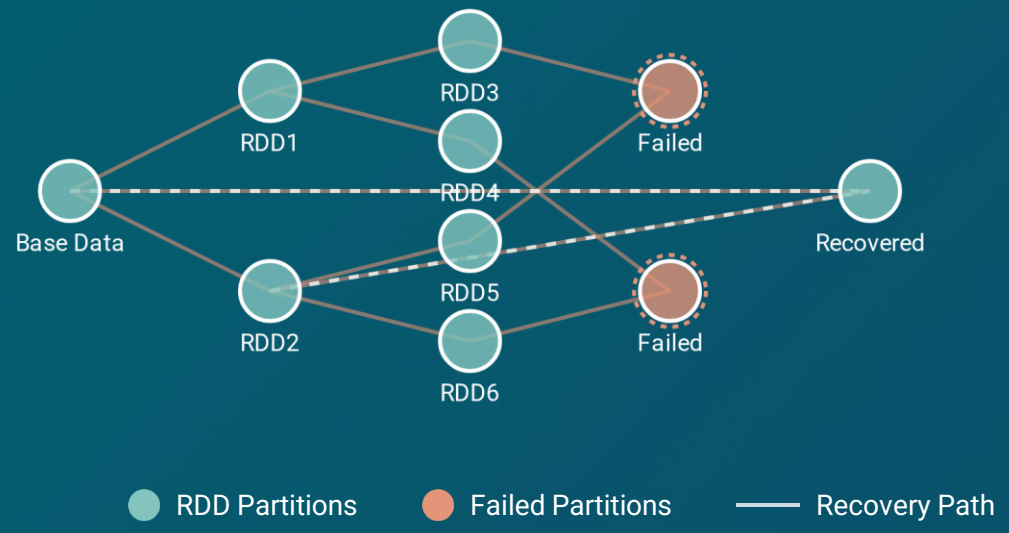
Efficient Recovery

In case of node failure, Spark leverages lineage information to recompute only the lost partitions, rather than reprocessing the entire dataset.

System Resilience

This mechanism ensures data recovery and system resilience without the need for costly data replication across the cluster.

Lineage-DAG Example: Data Recovery Process



Partitioning and Parallelism

What is Partitioning?

Partitioning divides RDD data into logical subsets that can be processed independently. Each partition is processed by a single task on a worker node.

Impact on Parallelism

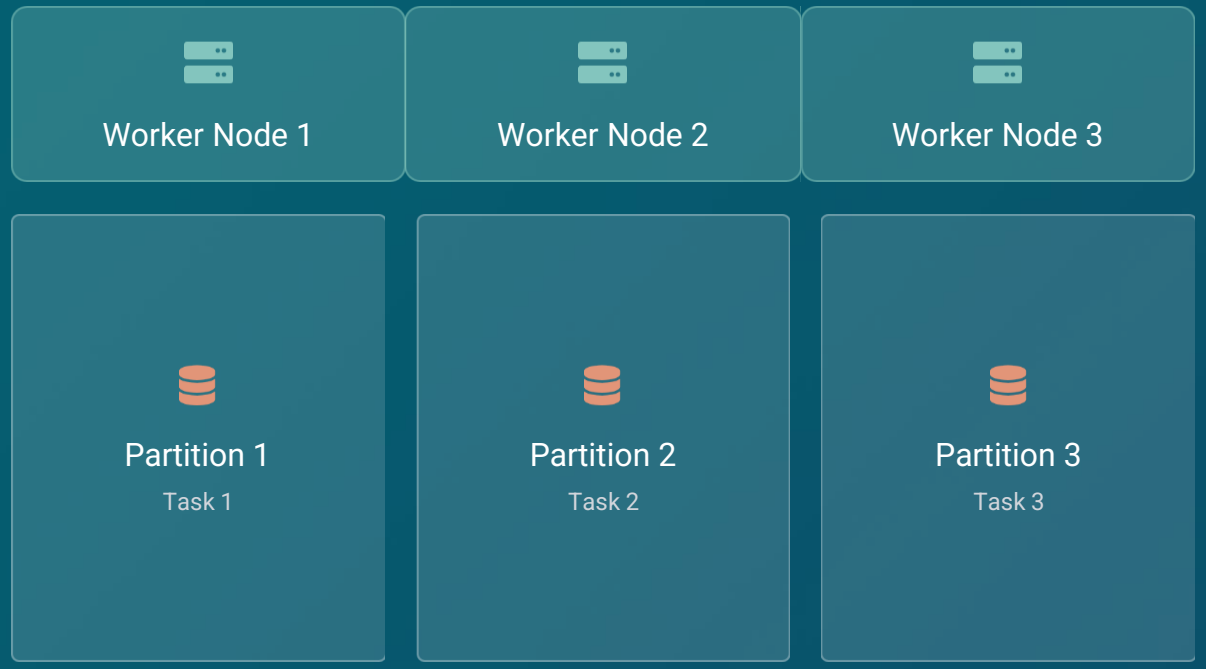
The number of partitions directly influences the degree of parallelism. More partitions allow more tasks to run concurrently, enhancing processing speed.

Setting Partitions

You can explicitly set the number of partitions when loading data:

```
rdd = sc.textFile("data.txt", minPartitions=100)
```

How Partitioning Works Across a Cluster



Key Benefits:

- Parallel Processing
- Resource Utilization
- Scalability
- Fault Isolation

Caching and Persistence

What is Caching?

Caching and persistence are optimization techniques designed to store RDD data in memory or on disk, facilitating rapid reuse in subsequent operations.

This significantly reduces computation overhead, especially for iterative algorithms or scenarios involving repeated access to the same dataset.

Example

```
rdd = sc.textFile("large_data.txt")
rdd.persist(StorageLevel.MEMORY_AND_DISK)

// Stores in memory, spills to disk if memory is insufficient
```

Benefits

- Reduced computation time for repeated operations
- Improved performance for iterative algorithms
- Enhanced throughput for data reuse scenarios

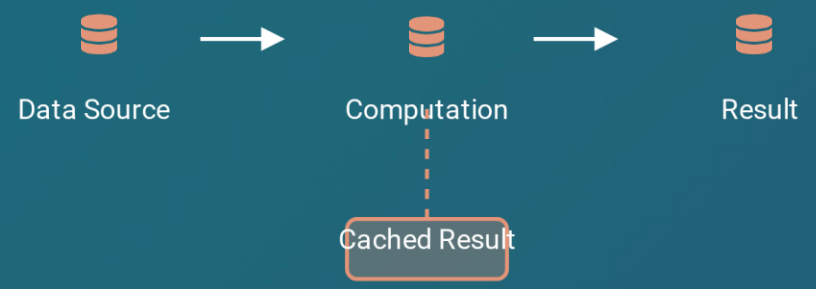
Storage Levels

MEMORY_ONLY
Store RDD in memory as Java objects

MEMORY_AND_DISK
Store in memory, spill to disk if insufficient

DISK_ONLY
Store only on disk

How Caching Improves Performance



Key-Based Operations

Key-Value Pair Operations

Apache Spark's RDDs offer powerful capabilities for processing key-value pairs (Pair RDDs), enabling efficient data aggregation and manipulation across distributed clusters.

```
# Sample sales data as key-value pairs
val rdd = sc.parallelize([("apple", 1), ("banana", 2), ("apple", 3)])
```

reduceByKey() Example

Aggregates values for each key by applying a associative function, performing local aggregations before shuffling data across the network.

```
val rdd_reduced = rdd.reduceByKey(_ + _)
# Output: [("apple", 4), ("banana", 2)]
```

Other Key-Based Operations

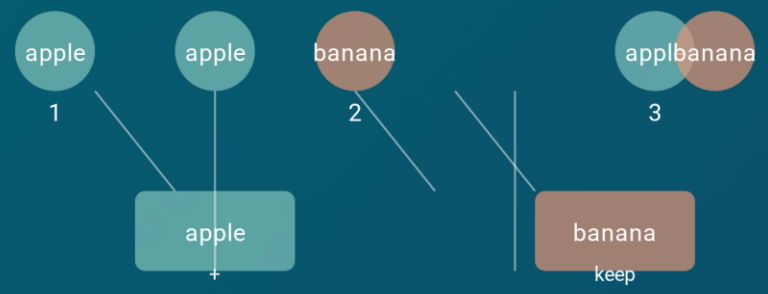
groupByKey()
Groups values for each key into an iterator

aggregateByKey()
More flexible aggregation with separate combiner

combineByKey()
Most flexible key-based operation

join()
Joins key-value pairs from two RDDs

How reduceByKey() Works



Conclusion and Best Practices

Key Takeaways

- RDDs provide a robust framework for handling large, distributed datasets through partitioning and parallel execution
- Fault tolerance through lineage-DAG ensures data recovery without costly data replication
- Key-based operations like reduceByKey enable efficient aggregation across distributed data
- Caching and persistence techniques optimize performance for iterative algorithms and repeated access

When to Use Spark RDDs

- Batch processing of large datasets that don't fit in memory
- Iterative algorithms requiring multiple passes over data
- Real-time data processing with fault tolerance requirements

Best Practices

Optimize Partitioning

Balance between parallelism and task overhead. Aim for 2-3 partitions per core.

Strategic Caching

Cache frequently accessed RDDs in memory with appropriate storage levels.

Minimize Data Shuffling

Use key-based operations to reduce data movement across the network.

Leverage Transformations

Use lazy evaluation to chain transformations and optimize execution plan.

RDD